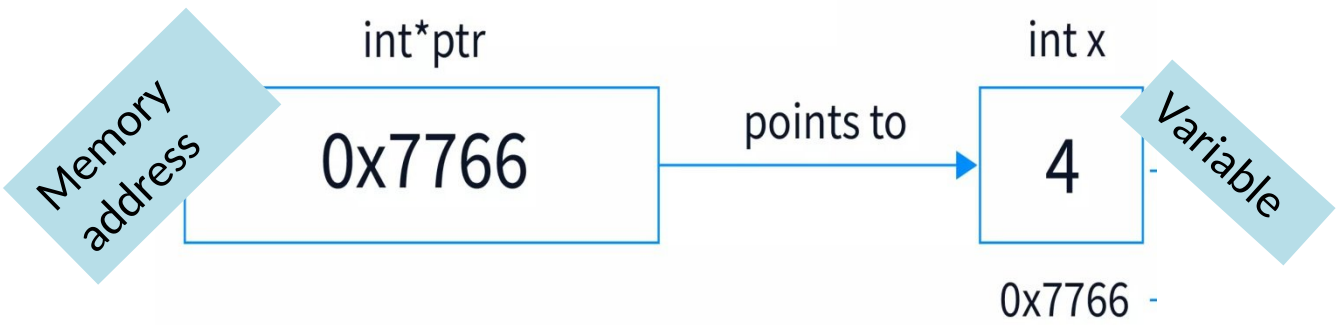
Module 4



Pointers

Part - I

Syllabus

- **Declaration, Operations on pointers**
- **Passing pointer to a function**
- **Accessing array elements using pointers**
- **Processing strings using pointers**
- **Pointer to pointer**
- Array of pointers
- Pointer to function
- Pointer to structure
- Dynamic Memory Allocation

Basics of Pointer

Definition: A pointer is a variable that stores the memory address of another variable

Declaration Syntax: `datatype * ptr;`

Example :

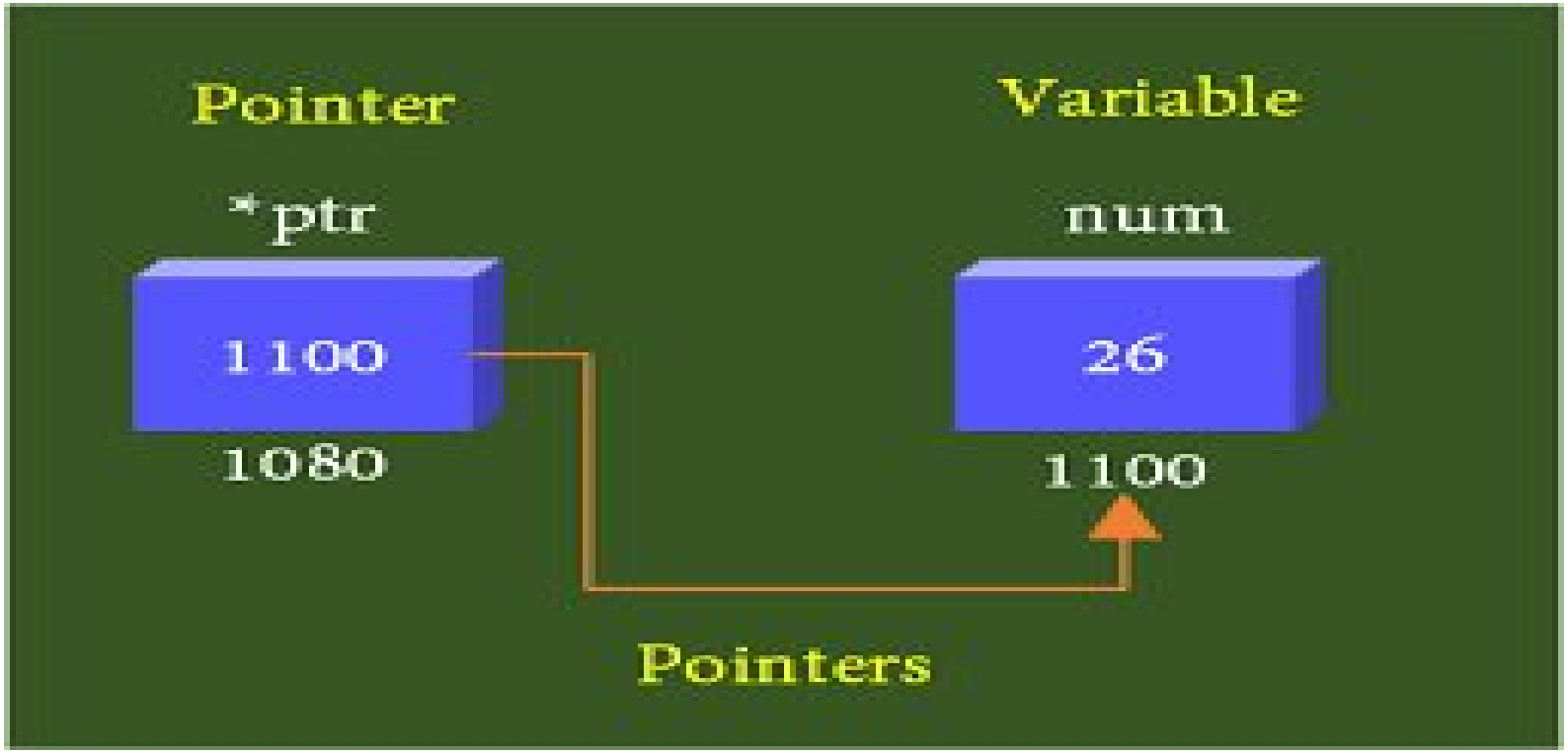
```
int *p1;    //to store the address of int variable
float *p2;  //to store the address of float variable
char *name; //to store the address of char variable
```

Three kinds of pointer declaration

```
int*    ptr
int     *  ptr
int    *ptr
```

Example

```
int   *pa, *pb, * p1;
```



*ptr = 26

ptr = 1100

num = 26

&num = 1100

Pointer = &Variable

Referred to as Pointer to variable (Pointer Variable)

Basics of Pointer

Initialization syntax:

```
datatype *ptr = &var;
```

Example:

```
int x;
int *ptr = &x;
```

Note : This is equivalent to

```
int x, *ptr;
ptr = &x;
```

Basics of Pointer

- `int` pointer can hold the address of `int` variable
- `float` pointer holds the address of a `float` variable.

& : Address operator

*** : Dereference operator (indirection operator)**

The operator & returns the address of the variable associated with it

- **pointer = & variable**
- **variable = * pointer**

Basics of Pointer

- ✓ Pointer is a derived data type
- ✓ It can be used to access and manipulate data stored in memory
- ✓ More efficient in handling arrays and tables
- ✓ Can return **multiple** values from a function
- ✓ Reduce the length and complexity of program
- ✓ Increase execution speed, reduce execution time

Pointer – program to print address

```
#include<stdio.h>
int main()
{
   int x;
   int *ptr1;
   x = 20;
   ptr1 = &x;
   printf("Value of x is : %d\n",x);
   printf("Value of *ptr1 is : %d\n",
*ptr1);
   printf("Value of ptr1 is : %p\n", ptr1);
   printf("Value of &x is : %p\n", &x);
   return 0;
```

Output:

Value of x is : 20

Value of ***ptr1*** is : 20

Value of ***ptr1*** is : 0x7ffd315ff43c

Value of ***&x*** is : 0x7ffd315ff43c

Pointer – program_2 to print address

```
#include<stdio.h>
int main()
{
   int x,y;
   int *ptr1,*ptr2;
   x = 20;
   ptr1 = &x;
   y = 30;
   ptr2 = &y;
   printf("Value of *ptr1 is : %d\n",
*ptr1);
   printf("Value of ptr1 is : %p\n", ptr1);
   printf("Value of *ptr2 is : %d\n",
*ptr2);
   printf("Value of
   return 0;
}
```

Address of x

Address of y

Output:

Value of *ptr1 is : 20

Value of ptr1 is : 0x7ffc29f5a970

Value of *ptr2 is : 30

Value of ptr2 is : 0x7ffc29f5a974

Memory Address range

64KB represents 65,536 bytes (8-bit) of memory

65,536 x 8 = 524,288 total bits .

0x0000 to 0xFFFF

0x**** represents hexadecimal equivalent of decimal addresses 0 to 65,535 (Total - 65,536 locations).

Binary	Hexadecimal	Decimal
0000 0000 0000 0000	0000	0
0000 0000 0000 0001	0001	1
⋮ ⋮ ⋮	⋮	⋮
1111 1111 1111 1110	FFFE	65534
1111 1111 1111 1111	FFFF	65535

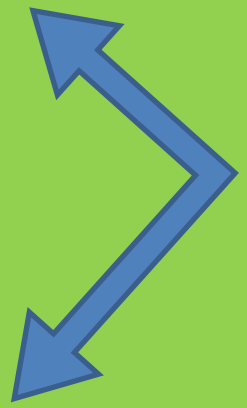
Program_2 - Output

Value of *ptr1 is : 20

Value of ptr1 is : 0x7ffc29f5a970

Value of *ptr2 is : 30

Value of ptr2 is : 0x7ffc29f5a974



Note : The difference is 4.
int x = 20 needs 4 memory location

Program - pointer

```
#include<stdio.h>
int main()
{
   char x = 'R';
   float y = 2.3;
   int z = 6;
   char *ptr1;
   float *ptr2;
   int *ptr3;
   ptr1 = &x;
   ptr2 = &y;
   ptr3 = &z;
```

```
printf(" *ptr1 is: %c\n",*ptr1);
printf(" ptr1 is : %p\n", ptr1);
printf("*ptr2 is: %f\n", *ptr2);
printf(" ptr2 is : %p\n", ptr2);
printf(" *ptr3 is: %d\n",*ptr3);
printf(" ptr3 is : %p\n", ptr3);
return 0;
}
```

Output	
*ptr1 is : R	ptr1 is : 0x7ffec05dd517
*ptr2 is : 2.300000	ptr2 is : 0x7ffec05dd518
*ptr3 is : 6	ptr3 is : 0x7ffec05dd51c

Recollect - **sizeof** - operator

```
#include<stdio.h>
int main()
{
char x = 'R';
float y = 2.3;
int z = 6;
printf("The size of Char is : %ld \n",sizeof(x));
printf("The size of Float is : %ld \n",sizeof(y));
printf("The size of Int is : %ld \n", sizeof(z));
return 0;
}
```

Output: (varies for machines)

The size of Char is : 1

The size of Float is : 4

The size of Int is : 4

Pointer increment and scale factor

```
#include<stdio.h>
int main()
{
int x, *ptr;
ptr = &x;
x = 400;
printf("Value of ptr is :%p\n", ptr);
printf("Value of ptr+1 is:%p\n", ptr+1);
return 0;
}
```

Output:

Value of ptr is : 0x7ffcc0a05d33

Value of **ptr+1** is : 0x7ffcc0a05d37

Pointer increment and scale factor

Scale factor:
When a pointer is incremented, its value is increased by the '*length*' of the data type that it points to. This length is called *scale factor*.

The number of bytes used to store various data types depends on the system.
It can be found by **sizeof** operator.

Programs - Pointer

```
#include<stdio.h>
int main()
{
int *ptr, c;
c = 5;
ptr = &c;
c = 12;
printf("The value of c is : %d\n", c);
printf("The value at address of ptr is : %d", *ptr);
return 0;
}
```

Output:

The value of c is : **12**

The value at address of ptr is : **12**

Programs - Pointer

```
#include<stdio.h>
int main()
{
int *ptr, c;
c = 5;
ptr = &c;
*ptr = 12;
printf("The value of c is : %d\n", c);
printf("The value at address of ptr is : %d", *ptr);
return 0;
}
```

Output:

The value of c is : **12**

The value at address of ptr is : **12**

WAP to find sum of two numbers using pointers

```
#include<stdio.h>
int main()
{
int a, b, *pa,*pb, sum;
pa = &a;
pb = &b;
a = 12; b = 34;
sum = *pa + *pb;
printf("The sum of two numbers is: %d",sum);
return 0;
}
```

Output:
The sum of two numbers is : 46

WAP to find the largest of two numbers using pointers

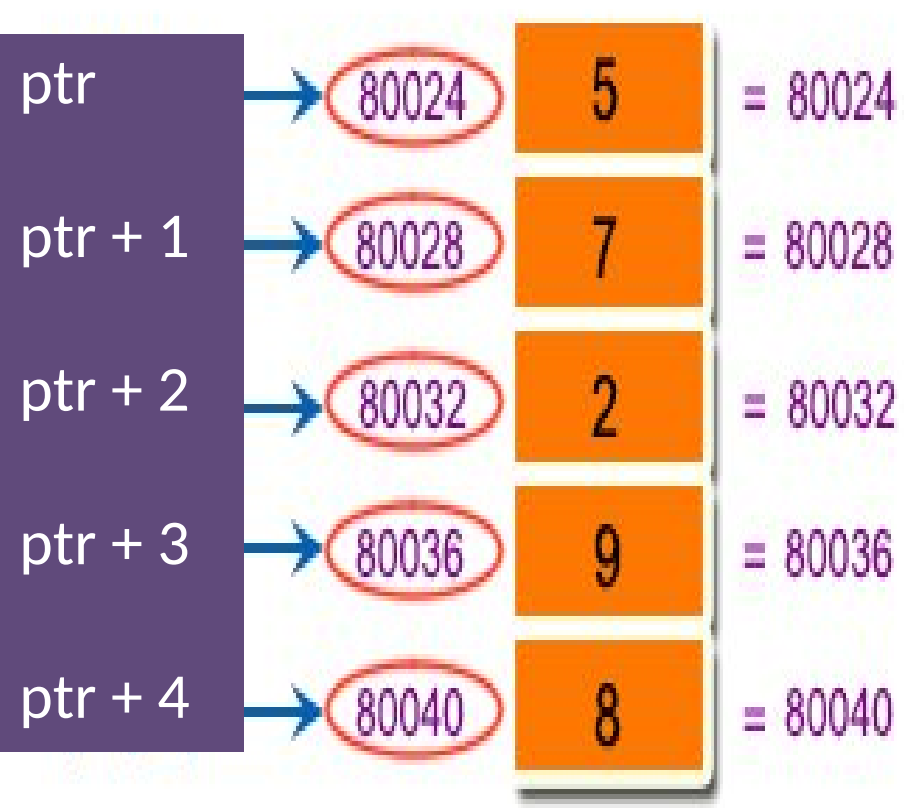
```
#include<stdio.h>
int main()
{
int a,b,*pa,*pb;
pa = &a;
pb = &b;
a = 45; b = 67;
if(*pa > *pb)
   printf("The largest number is : %d", *pa);
else
   printf("The largest number is : %d", *pb);
return 0;
}
```

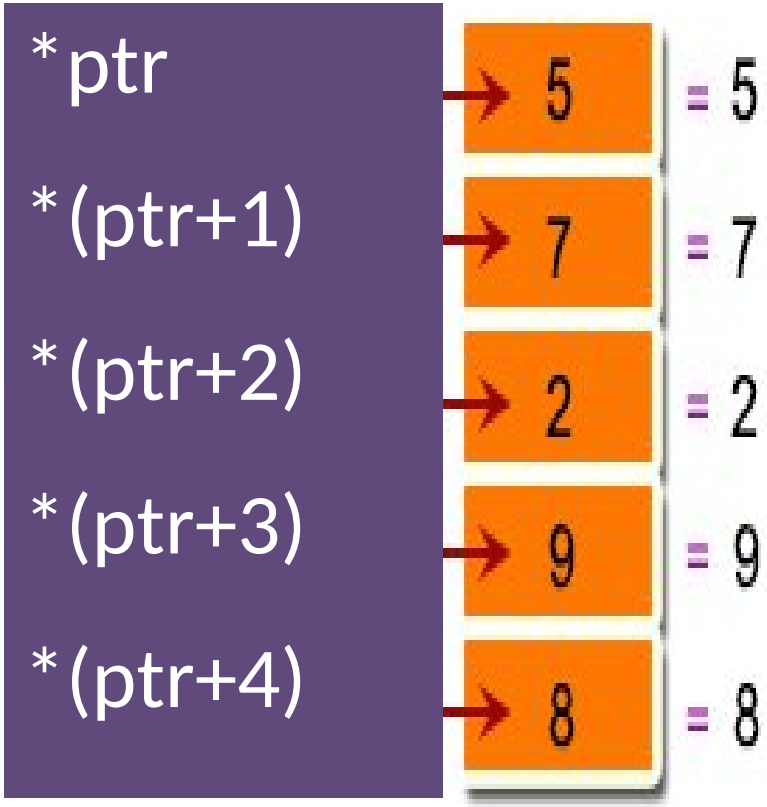
Output:

The largest number is : 67

Accessing array elements using pointers


ptr
arr [0]
arr [1]
arr [2]
arr [3]
arr [4]
80024
5
7
2
9
8
105320
80024
80028
80032
80036
80040


ptr
ptr + 1
ptr + 2
ptr + 3
ptr + 4
80024
80028
80032
80036
80040
5
7
2
9
8
= 80024
= 80028
= 80032
= 80036
= 80040


*ptr
*(ptr+1)
*(ptr+2)
*(ptr+3)
*(ptr+4)
5
7
2
9
8
= 5
= 7
= 2
= 9
= 8

Pointer - arrays

- When an array is declared, the compiler allocates a **base address** and sufficient amount of memory to contain all elements
- The base address is the location of first element – array_name[0]
- Thus for array marks[]

  pointer to array is **`ptr = marks;`**

  equivalent to `ptr = &marks[0];`

Relation between ptr and x

Pointer	Address - Array-Index	Address
ptr	&x[0]	1100
ptr + 1	&x[1]	1104
ptr + 2	&x[2]	1108
ptr + 3	&x[3]	110C
ptr + 4	&x[4]	1110

```
scanf("%d", &x[i]);      can replaced by
scanf("%d", ptr + i); // '&' not needed
```

Pointer	Address	Address - Array Index
p	1100	&x[0]
p + 1	1104	&x[1]
p + 2	1108	&x[2]
p + 3	110C	&x[3]
p + 4	1110	&x[4]

Array Elements
20
30
40
50
60

p	1100
p + 3	110c

*p	20
*p + 3	**23**
*(p+3)	**50**

WAP to input and print the elements of an array using pointers.

```
int x[50],i, N, *ptr;
ptr = x;
printf("Enter the size of the array : ");
scanf("%d", &N);
printf("Enter the elements of the array:\n");
for (i=0; i<N ; i++)
  scanf("%d", ptr + i);  // no need '&'
printf("The given array is : \n");
for (i=0; i<N ; i++)
 {
   printf("%d\n", *(ptr + i));
 }
```

Output

Enter the size of the array : 5 Enter the elements of the array: 14 26 31 44 58	The given array is : 14 26 31 44 58

Base address of array

```
#include<stdio.h>
int main()
{
int x[50];
printf("The address of array begins at:\n");
printf("%p \n %p \n %p", x, &x, &x[0]);
return 0;
}
```

The name of the array can be used as a pointer

Output:

The address of array begins at :

0x7ffd80d3b7c0

0x7ffd80d3b7c0

0x7ffd80d3b7c0

WAP to display the array elements with **array name** as pointer

```
#include <stdio.h>
int main()
{
int x[50], i, N;
printf("Enter the size of array: ");
scanf("%d", &N);
printf("Enter the elements of the array:\n");
for(i = 0; i < N; i++)
      scanf("%d", x+i);
printf("The array elements are :\n");
for(i = 0; i < N; i++)
       printf("%d\n", *(x+i));
return 0;
}
```

Output

Enter the size of the array : 5 Enter the elements of the array: 14 26 31 44 58	The array elements are : 14 26 31 44 58

WAP to find the sum of the elements of an array using pointers.

```
int i, N, x[50], sum, *p;
sum = 0;
p = x;
printf("Enter the size of array : ");
scanf("%d", &N);
printf("The elements are : ");
for(i=0; i < N; i++)
   {
      scanf("%d", p+i);
      sum = sum + *(p+i);
   }
printf("The sum is:%d",sum);
```

Output:

Enter the size of array : 5

The elements are :

10

20

30

40

50

The sum is : 150

WAP to find the smallest element in an array using pointers.

```
#include<stdio.h>
int main()
{
  int i, N, x[50], *p;
  p = x;
  printf("Enter the size of array : ");
  scanf("%d", &N);
  printf("The elements are : ");
  for(i=0; i < N; i++)
  {
    scanf("%d", p+i);
```

Smallest element – pointer- array

```
for(i=1; i<N; i++)
    {
    if(*p > *(p+i))
        {
        *p = *(p+i);
        }
    }
printf("The smallest element is :%d", *p);
return 0;
}
```

Output

Enter the size of array : 4

The elements are :

34

56

0

-9

The smallest element is : -9

Note: The first element *p = x[0] is assumed to be smallest during start; so the “for loop” starts with ‘1’

Processing strings using pointers

Pointer to string

Declaration syntax

```
char  *ptr
```

Initialization syntax

```
char *ptr  = &str
```

or

If declared , initializing can be done as

```
char  str [20], *ptr
ptr = str
```

Pointer to string

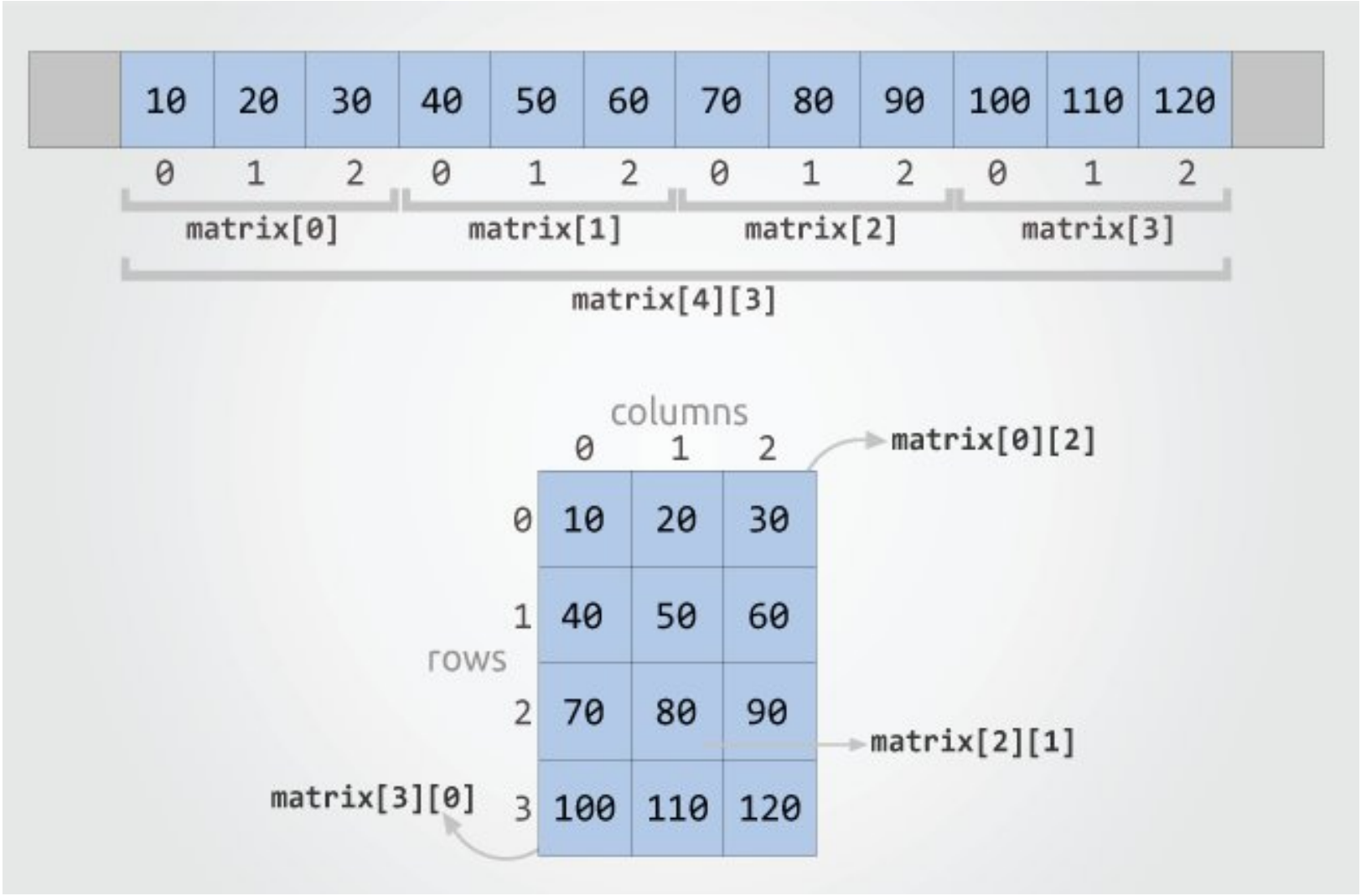
```
#include<stdio.h>
int main()
{
char ch,str[50],
*ptr;
int length;
ptr = str;
printf("Enter a word :");
scanf("%s", ptr);
printf("The given word is :
\n");
 while(*ptr != '\0')
    {
    printf("%c\n", *ptr);
    ptr++;
    }
 length = ptr - str;
 printf("The length of the
    string is : %d", length);
 return 0;
}
```

Output

```
Enter a word : Lucifer
The given word is :
L
u
c
i
f
e
r
The length of the string is : 7
```

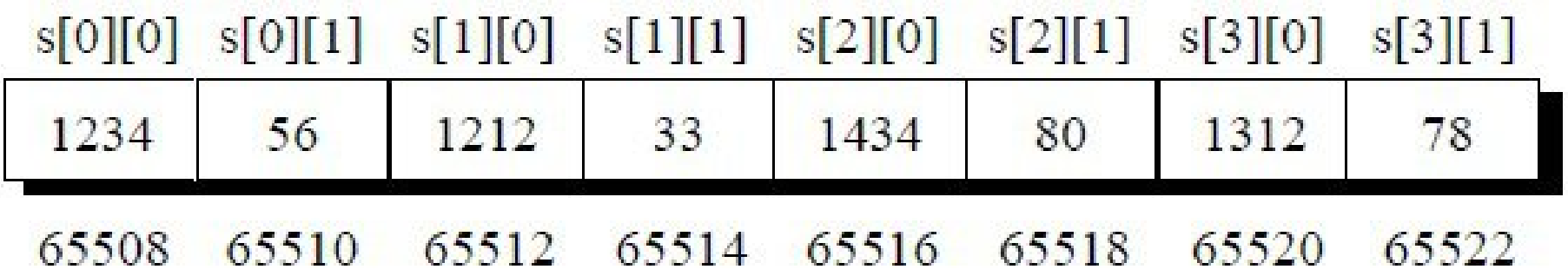
Pointer to Two dimensional array

Two dimensional array in memory

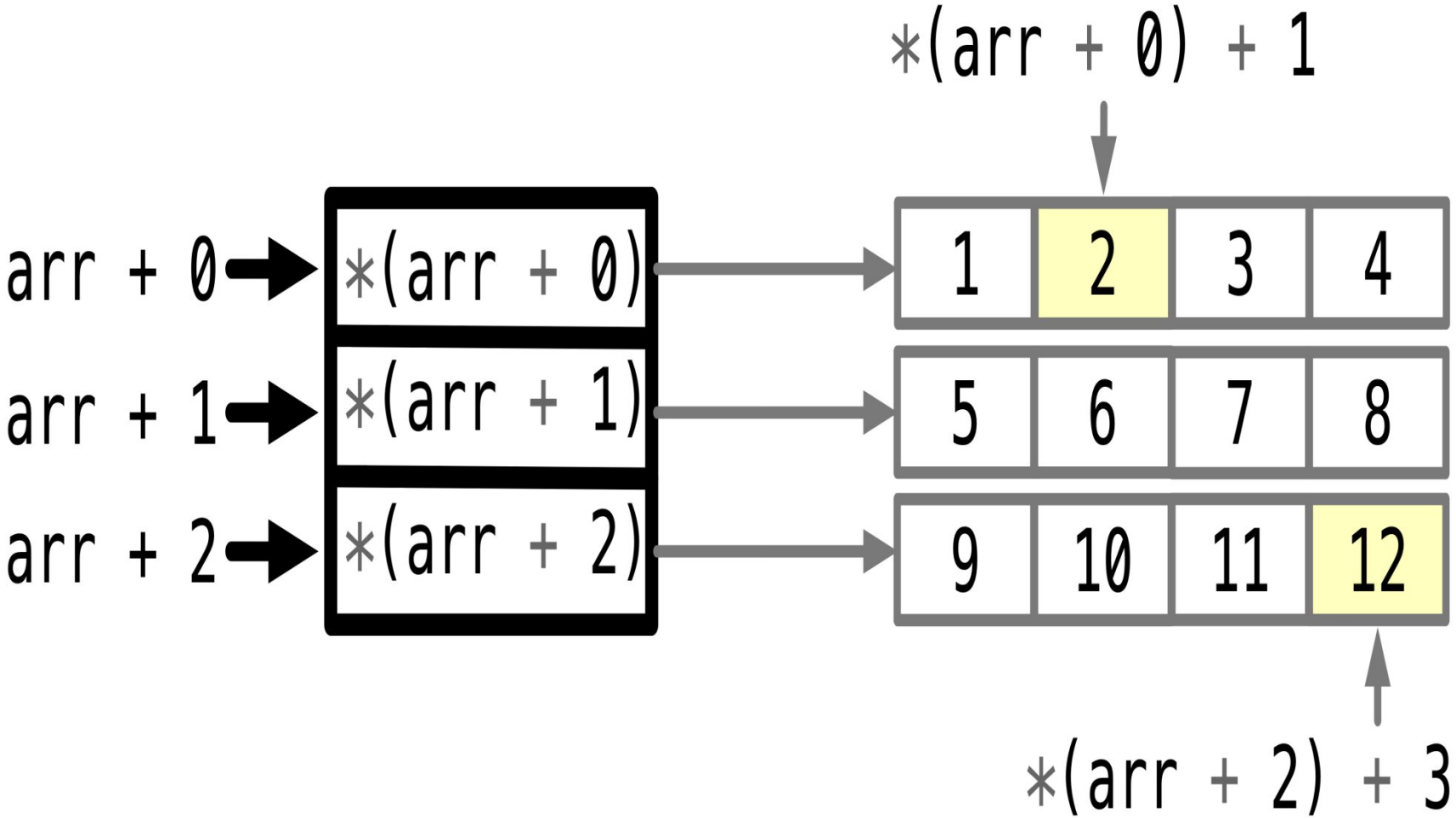


Memory Map of a 2-Dimensional Array in C

Memory doesn't contain rows and columns.
In memory, the array elements are stored in one continuous chain.
Example – 4X2 matrix



Dynamic allocation of an array of pointers



Pointer representation for 2-D

	One dimensional array	Two dimensional array
Elements	arr[i]	mat[i][j]
Pointer	arr = ptr	ptr – pointer to first row
	ptr+i	ptr+i - pointer to i^{th} row
		(ptr+i)+j - pointer to i^{th} row, j^{th} element

Pointer syntax for 2 - D

Declaration syntax

```
datatype *ptr
```

Initialization syntax

```
datatype *ptr = &arr[0][0]
```

Input and display a **m x n** matrix using pointers

```
#include<stdio.h>
int main()
{
int i, j, m, n, mat[10][20], *ptr;
ptr = &mat[0][0];
printf("Enter number of rows :  ");
scanf("%d", &m);
printf("Enter number of columns : ");
scanf("%d", &n);
```

Input and display a **m x n** matrix using pointers - contd

```
for(i = 0; i < m; i++)
{
 for (j = 0; j < n; j++)
 {
  printf("Enter value of mat[%d][%d]: ",i,j);
  scanf("%d", (ptr+i)+j);
 }
}
```

Input and display a **m x n** matrix using pointers - contd

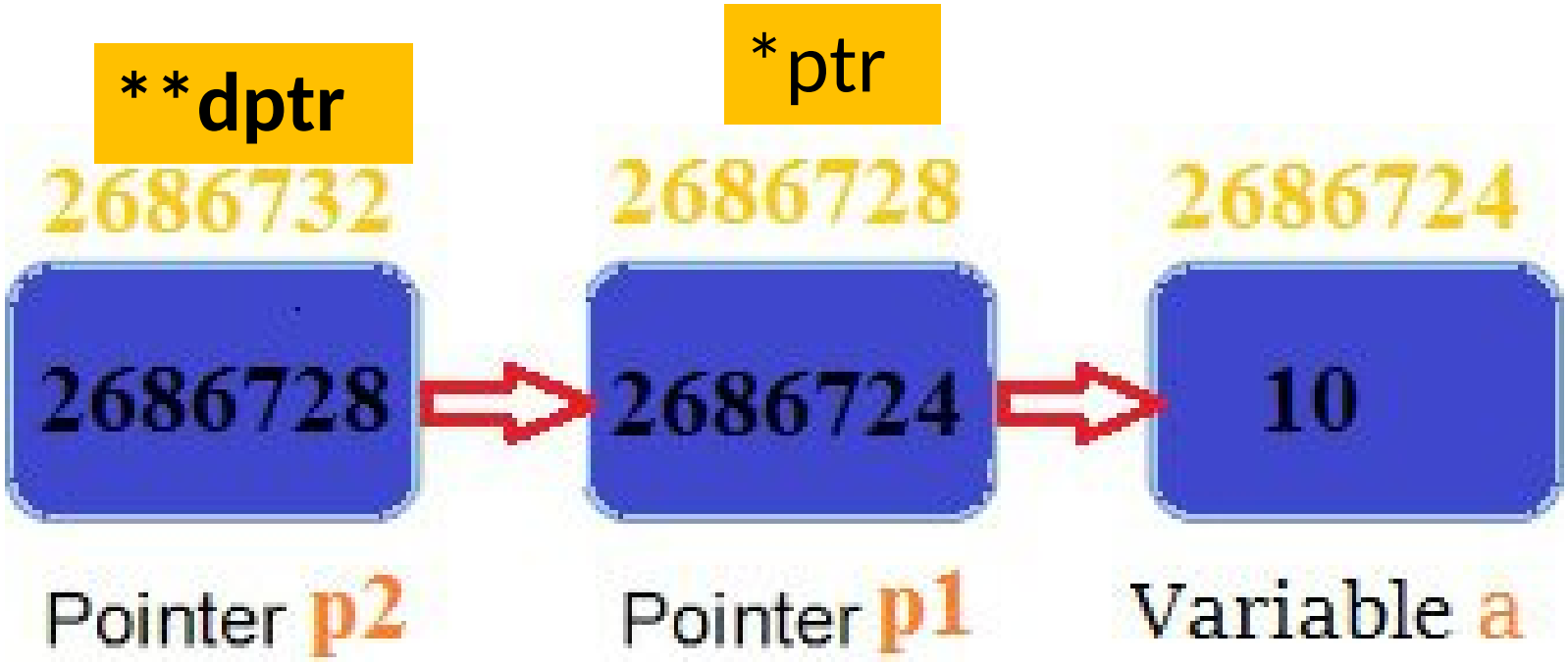
```
printf("The matrix is given below :\n");
for (i = 0; i < m; i++)
   {
   for (j = 0; j < n; j++)
      {
        printf("%d\t", *(ptr+i)+j);
      }
   printf("\n");
   }
return 0;
}
```

Output

```
Enter number of rows : 3
Enter number of columns : 2
Enter value of mat[0][0]: 1
Enter value of mat[0][1]: 2
Enter value of mat[1][0]: 3
Enter value of mat[1][1]: 4
Enter value of mat[2][0]: 5
Enter value of mat[2][1]: 6
```

```
The matrix is given below :
        1    2
        3    4
        5    6
```

Pointer to pointer



Declaration Syntax:
data_type **variable_name

Pointer to pointer

- A pointer variable can store the address of any datatype
  - primary data types - int, float, char,
  - derived data types - arrays, structure

- Pointer can store the address of another pointer too, called →pointer to pointer→ (**double pointer**).

WAP using double pointer

```
#include<stdio.h>
int main()
{
 int x, *ptr, **dptr;
 x = 20;
 ptr = &x;
 dptr = &ptr;
 printf("%d\n %p \n %p ", x, ptr, dptr);
 return 0;
}
```

Output:

20

0x7ffde26b38b4

0x7ffde26b38b8